

프로그래밍응용개념

#06 클래스

국립한밭대학교 인공지능 소프트웨어학과

이성민

클래스

- 객체지향 프로그래밍
- 클래스
- 싱글톤 패턴

클래스

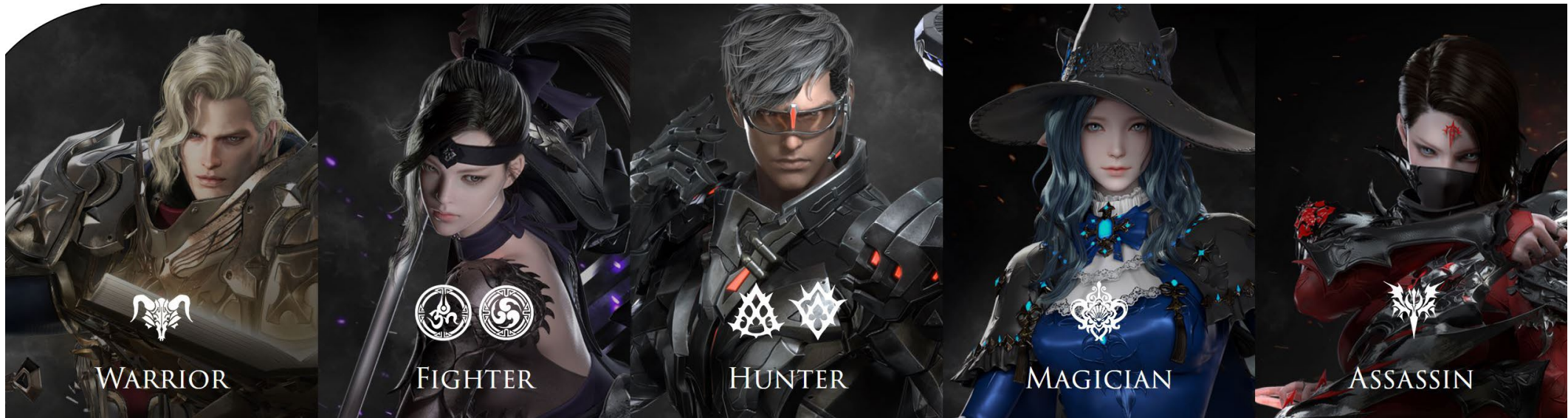
- 클래스와 인스턴스

- 클래스: 객체를 생성하기 위한 설계도 (객체가 무엇인지 정의)
- 인스턴스: 클래스로부터 생성된 객체
- 인스턴스화: 클래스로부터 객체를 만드는 과정
- 하나의 클래스로부터 여러 개의 인스턴스를 만들 수 있음



객체 지향 프로그래밍

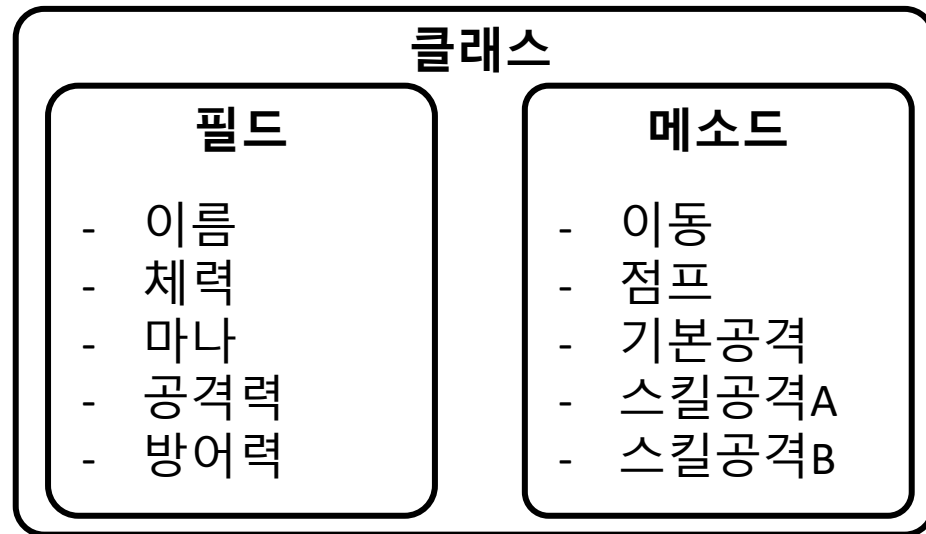
- 클래스 (= 직업)
- 객체 (인스턴스) (= 캐릭터)



객체 지향 프로그래밍

- 객체 (Object)

- 객체란 다른 것과 식별 가능한 것
- 객체는 속성과 동작으로 구성 (속성 = 필드, 동작 = 메소드)



→ 생성



객체 지향 프로그래밍

- 객체 지향 프로그래밍 (Object-Oriented Programming)
 - 객체들을 먼저 만들고, 이 객체들을 하나씩 조립해서 완성된 프로그램을 만드는 기법
- 객체의 상호 작용
 - 각 객체들은 다른 객체와 서로 상호작용하면서 동작
 - 다른 객체의 기능을 이용할 때, 메소드를 호출해 데이터를 주고 받음

객체 지향 프로그래밍

- 절차 지향 프로그래밍 (Procedural Programming)
 - 순차적인 프로그램 처리 방식, 각 작업을 독립적인 함수로 분리.

객체 지향 프로그래밍

- 객체 간의 관계
 - 집합 관계: 완제품과 부품



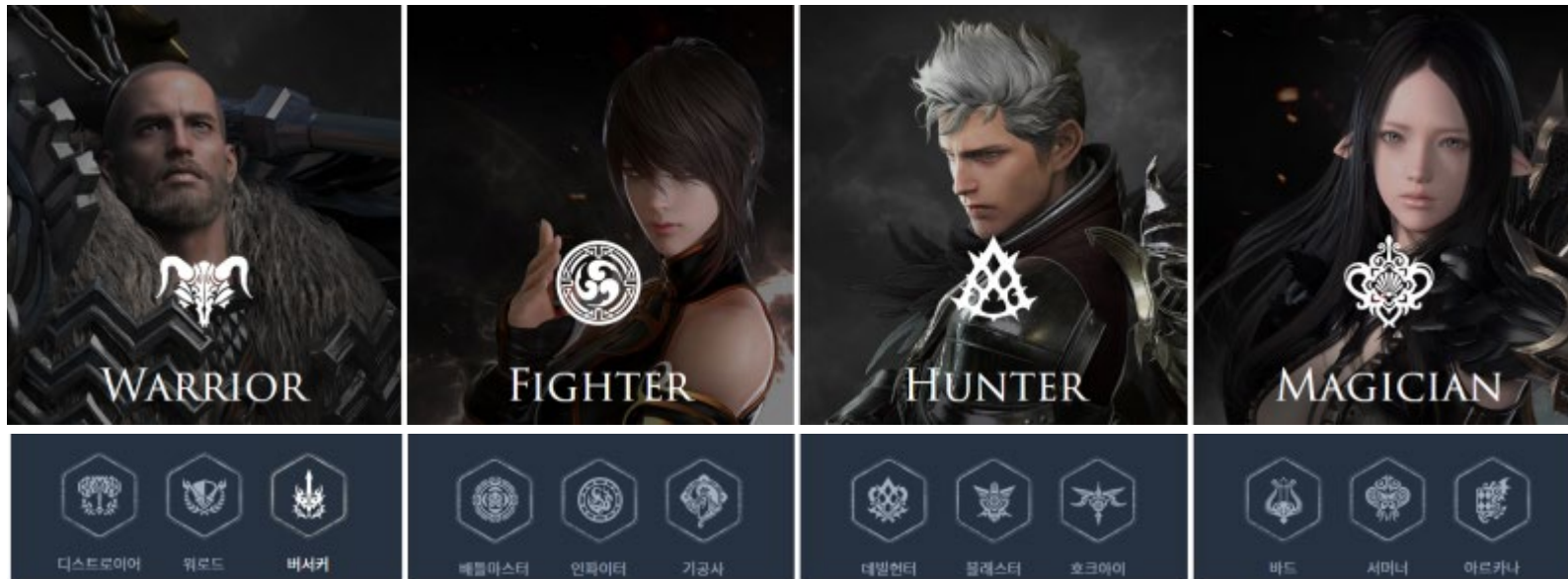
객체 지향 프로그래밍

- 객체 간의 관계
 - 사용 관계: 다른 객체의 필드를 읽고 변경하거나, 메소드를 호출



객체 지향 프로그래밍

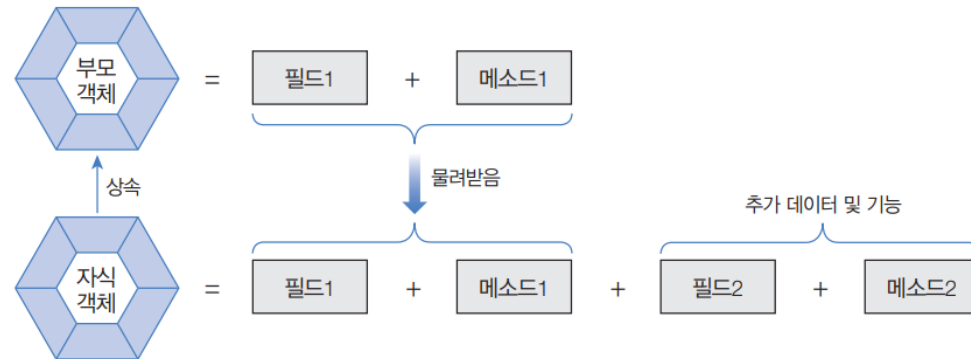
- 객체 간의 관계
 - 상속 관계: 부모와 자식 관계 (필드, 메소드를 물려받음)



객체 지향 프로그래밍

- 객체 지향 프로그래밍의 특징

- 상속: 부모 객체가 자기 필드와 메소드를 자식 객체에게 물려줘 자식 객체가 사용할 수 있게 함

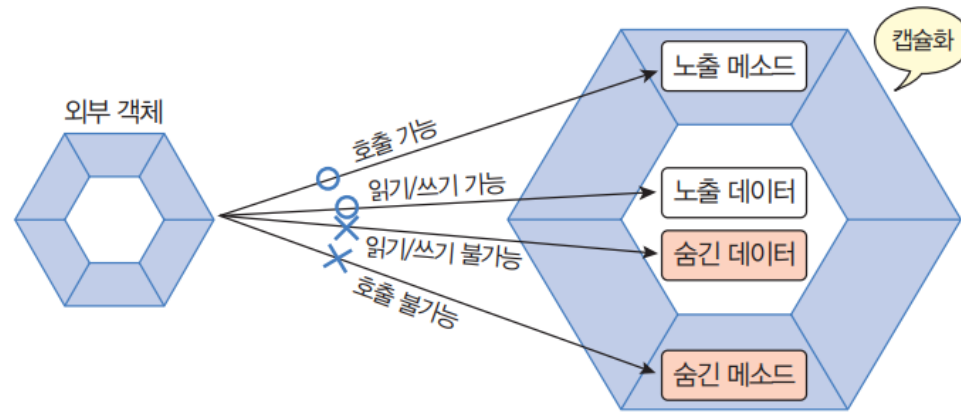


- 다형성: 하나의 메소드나 변수가 다양한 동작 방식을 가질 수 있음

객체 지향 프로그래밍

- 객체 지향 프로그래밍의 특징

- 캡슐화: 객체의 데이터 (필드), 동작 (메소드)을 하나로 묶고 실제 구현 내용을 외부에 감추는 것



클래스

- 클래스 선언

- 객체를 생성(생성자)하고, 객체가 가져야 할 데이터(필드)가 무엇이고, 객체의 동작(메소드)은 무엇인지를 정의
- 클래스 선언은 소스 파일명과 동일하게 작성
 - 클래스명은 첫 문자를 대문자로 하고 camel 스타일로 작성. 숫자를 포함해도 되지만 첫 문자는 숫자가 될 수 없고, 특수 문자 중 \$, _를 포함할 수 있음

[클래스명.java]

```
//클래스 선언  
public class 클래스명 {  
}
```

클래스

- 클래스의 구성 멤버
 - 필드: 객체의 데이터를 저장하는 역할
 - 메소드: 객체가 수행할 동작
 - 생성자: new 연산자로 객체를 생성할 때, 객체의 초기화 역할

- 필드
객체의 데이터가 저장되는 곳
- 생성자
객체 생성 시 초기화 역할 담당
- 메소드
객체의 동작으로 호출 시 실행하는 블록

```
public class ClassName {  
    //필드 선언  
    int fieldName;  
  
    //생성자 선언  
    ClassName() { ... }  
  
    //메소드 선언  
    int methodName() { ... }  
}
```

클래스

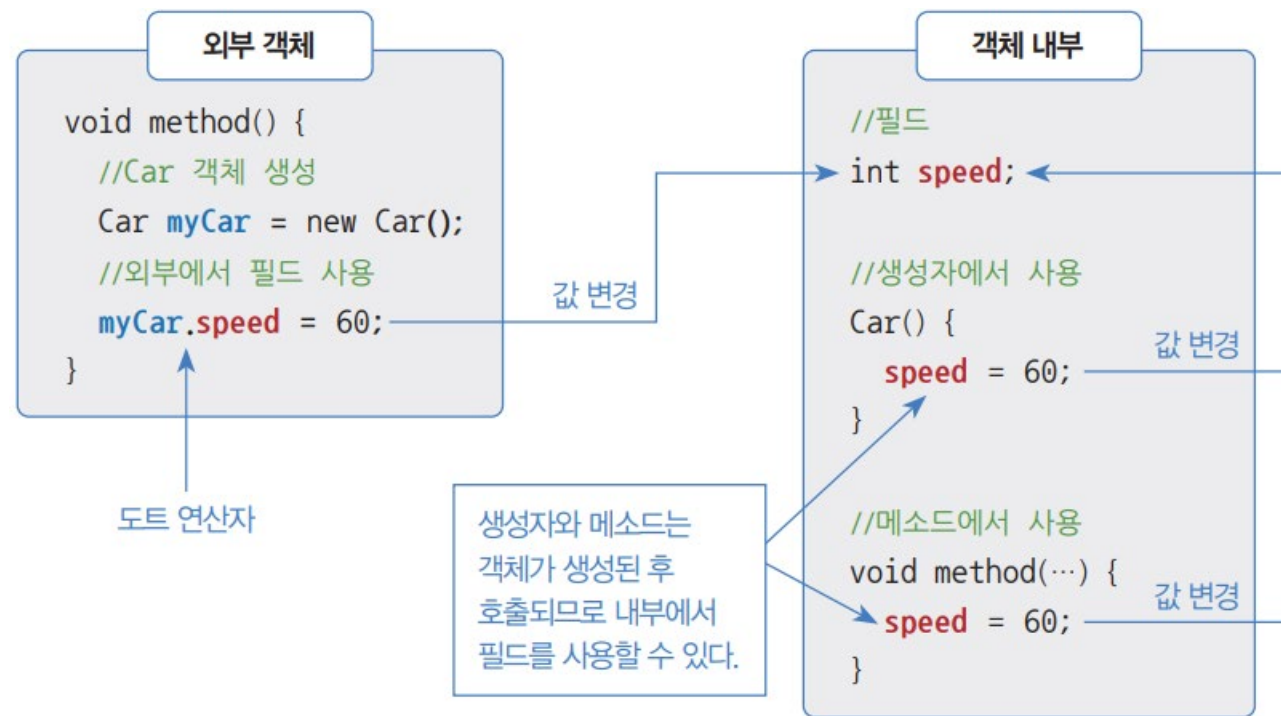
- 필드 선언

- 필드는 클래스 블록에서 선언되어야 함
- 필드의 타입을 지정해줘야 함
- 초기값을 제공하지 않는 경우는 객체 생성시 기본값으로 초기화

분류		데이터 타입	기본값
기본 타입	정수 타입	byte char short int long	0 \u0000 (빈 공백) 0 0 0L
	실수 타입	float double	0.0F 0.0
	논리 타입	boolean	false
참조 타입		배열 클래스(String 포함) 인터페이스	null null null

클래스

- 필드 사용
 - 필드 값을 읽고 변경 가능
 - 객체가 생성된 후에 필드를 사용할 수 있음
 - 객체의 내외부에서 접근 가능



클래스

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
Main.java
1
2
3 public class Main {
4     public static void main(String[] args) {
5         Car myCar = new Car();
6
7         myCar.model = "제네시스";
8         myCar.maxSpeed = 250;
9         myCar.start = false;
10
11         System.out.println("최대속도: " + myCar.maxSpeed);
12         System.out.println("현재 시동 상태: " + myCar.start);
13
14         myCar.setStart();
15         System.out.println("현재 시동 상태: " + myCar.start);
16     }
17 }

Car.java
1
2
3 public class Car { 2개 사용 위치
4     String model; 1개 사용 위치
5     int maxSpeed; 2개 사용 위치
6     boolean start; 4개 사용 위치
7
8     void setStart() { 1개 사용 위치
9         start = true;
10     }
11 }
12
```

클래스

- 생성자 선언

- 객체를 다양하게 초기화하기 위해 생성자를 직접 선언할 수 있음

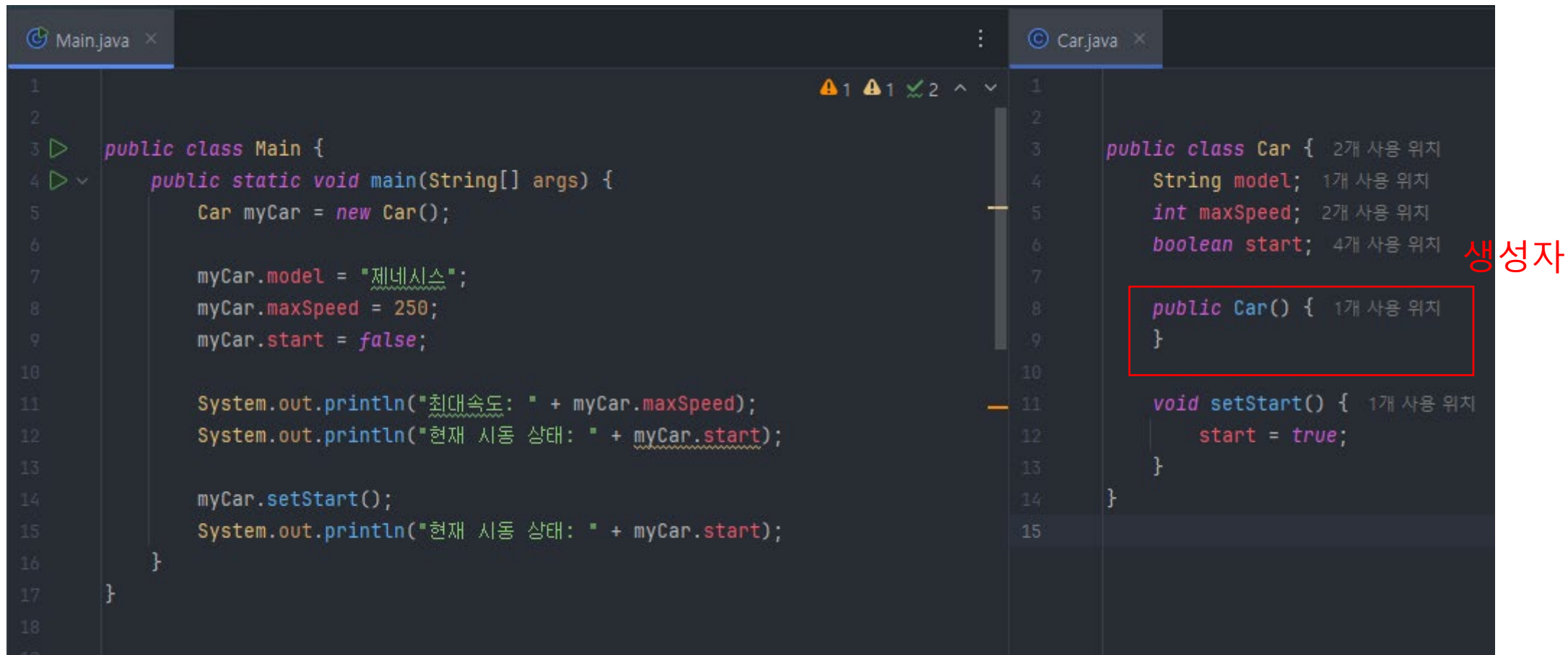
```
클래스(매개변수, ... ) {  
    //객체의 초기화 코드  
} } 생성자 블록
```

- 생성자 선언과 호출

- 기본 생성자: 클래스에 생성자 선언이 없으면 컴파일러는 기본 생성자를 바이트코드 파일에 자동으로 추가

```
[public] 클래스() { }
```

클래스



```
1  public class Main {
2
3  public static void main(String[] args) {
4      Car myCar = new Car();
5
6      myCar.model = "제네시스";
7      myCar.maxSpeed = 250;
8      myCar.start = false;
9
10     System.out.println("최대속도: " + myCar.maxSpeed);
11     System.out.println("현재 시동 상태: " + myCar.start);
12
13     myCar.setStart();
14     System.out.println("현재 시동 상태: " + myCar.start);
15 }
16 }
17
18
19
```

```
1  public class Car { 2개 사용 위치
2
3      String model; 1개 사용 위치
4      int maxSpeed; 2개 사용 위치
5      boolean start; 4개 사용 위치
6
7      public Car() { 1개 사용 위치
8      }
9
10     void setStart() { 1개 사용 위치
11         start = true;
12     }
13 }
14
15
```

생성자

클래스

- 필드 초기화
 - 객체마다 동일한 값을 갖고 있다면 필드 선언 시 초기값 대입 권장
 - 객체마다 다른 값을 가져야 한다면 생성자에서 필드 초기화 권장

클래스

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car(model: "제네시스", maxSpeed: 250, start: false);  
  
        System.out.println("최대속도: " + myCar.maxSpeed);  
        System.out.println("현재 시동 상태: " + myCar.start);  
  
        myCar.setStart();  
        System.out.println("현재 시동 상태: " + myCar.start);  
    }  
}
```

```
3 public class Car { 2개 사용 위치  
4     String model; 2개 사용 위치  
5     int maxSpeed; 3개 사용 위치  
6     boolean start; 5개 사용 위치  
7  
8     public Car(String model, int maxSpeed, boolean start) {  
9         this.model = model;  
10        this.maxSpeed = maxSpeed;  
11        this.start = start;  
12    }  
13  
14    void setStart() { 1개 사용 위치  
15        start = true;  
16    }  
17 }
```

생성자

필드 초기화

클래스

- this 키워드
 - 객체 내부에서는 인스턴스 멤버에 접근하기 위해 this 사용
 - 객체는 자신을 this로 지칭
 - 생성자와 메소드의 매개변수명이 인스턴스 멤버인 필드명과 동일한 경우, 인스턴스 필드임을 강조하고자 할 때 this를 주로 사용

클래스

- 생성자 오버로딩 (Overloading)
 - 매개변수를 달리하는 생성자를 여러 개 선언하는 것
 - 오버로딩되어 있을 경우, new 연산자로 생성자를 호출할 때 제공되는 매개값의 타입과 수에 따라 실행될 생성자가 결정

```
public class Car {  
    Car() { ... }  
    Car(String model) { ... }  
    Car(String model, String color) { ... }  
    Car(String model, String color, int maxSpeed) { ... }  
}
```

[생성자 오버로딩]
매개변수의 타입, 개수, 순서가
다르게 여러 개의 생성자 선언

클래스

```
Main.java x
1
2
3 ▶ public class Main {
4 ▶ ~ public static void main(String[] args) {
5     Car myCar1 = new Car();
6
7     myCar1.model = "제네시스";
8     myCar1.maxSpeed = 250;
9     myCar1.start = false;
10
11     Car myCar2 = new Car( model: "소나타", maxSpeed: 220);
12
13     System.out.println("최대속도: " + myCar1.maxSpeed);
14     System.out.println("최대속도: " + myCar2.maxSpeed);
15 }
16 }
17
18
19
20
21
22
23
24
25

Car.java x
1
2
3 public class Car { 4개 사용 위치
4     String model; 3개 사용 위치
5     int maxSpeed; 4개 사용 위치
6     boolean start; 5개 사용 위치
7
8     public Car(String model, int maxSpeed, boolean start) {
9         this.model = model;
10        this.maxSpeed = maxSpeed;
11        this.start = start;
12    }
13
14    public Car(String model, int maxSpeed) { 0개의 사용위치
15        this.model = model;
16        this.maxSpeed = maxSpeed;
17    }
18
19    public Car() { 1개 사용 위치
20    }
21
22    void setStart() { 1개 사용 위치
23        start = true;
24    }
25 }
```


클래스

- 다른 생성자 호출
 - 생성자 간의 중복된 부분이 발생하는 경우, 하나의 생성자만 집중 작성하고, 나머지 생성자는 `this` 를 사용해 공통 코드를 가진 생성자를 호출

클래스

```
Main.java x
1
2
3 public class Main {
4     public static void main(String[] args) {
5         Car myCar1 = new Car();
6
7         myCar1.model = "제네시스";
8         myCar1.maxSpeed = 250;
9         myCar1.start = false;
10
11         Car myCar2 = new Car(model: "소나타", maxSpeed: 220);
12         Car myCar3 = new Car();
13
14         System.out.println("차종: " + myCar1.model);
15         System.out.println("차종: " + myCar2.model);
16         System.out.println("차종: " + myCar3.model);
17     }
18 }
19
20
21
22
23
24
25

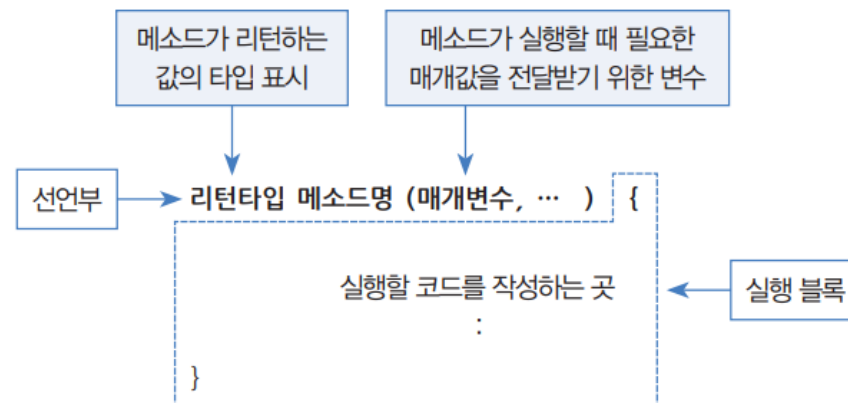
Car.java x
1
2
3 public class Car { 6개 사용 위치
4     String model; 2개 사용 위치
5     int maxSpeed; 3개 사용 위치
6     boolean start; 3개 사용 위치
7
8     public Car(String model, int maxSpeed, boolean start) {
9         this.model = model;
10        this.maxSpeed = maxSpeed;
11        this.start = start;
12    }
13
14    public Car(String model, int maxSpeed) { 1개 사용 위치
15        this(model, maxSpeed, start: false);
16    }
17
18    public Car() { 2개 사용 위치
19        this(model: "아반떼", maxSpeed: 200, start: false);
20    }
21
22    void setStart() { 0개의 사용 위치
23        start = true;
24    }
25 }
```

클래스

- 메소드 선언

- 메소드: 객체의 동작을 실행 블록으로 정의

- 메소드 명: 첫 문자를 소문자로 작성하고, camel 스타일로 작성
 - 리턴 타입: 메소드 실행 후 호출한 곳으로 전달하는 결과값의 타입
 - 매개변수: 메소드를 호출할 때 전달한 매개값을 받기 위해 사용



클래스

- 메소드 호출
 - 메소드 블록을 실제로 실행하는 것
 - 외부 객체에서는 참조 변수와 도트 (.) 연산자로 호출

클래스

```
Main.java x
1
2
3 ▶ public class Main {
4 ▶     public static void main(String[] args) {
5         Car myCar1 = new Car();
6
7         myCar1.model = "제네시스";
8         myCar1.maxSpeed = 250;
9         myCar1.start = false;
10
11         Car myCar2 = new Car( model: "소나타", maxSpeed: 220);
12         Car myCar3 = new Car();
13
14         System.out.println("차종: " + myCar1.model);
15         System.out.println("차종: " + myCar2.model);
16         System.out.println("차종: " + myCar3.model);
17
18         myCar2.setStart();
19
20         System.out.println("시동상태: " + myCar2.start);
21     }
22 }
23
24
25

Car.java x
1
2
3 public class Car { 6개 사용 위치
4     String model; 5개 사용 위치
5     int maxSpeed; 2개 사용 위치
6     boolean start; 3개 사용 위치
7
8     public Car(String model, int maxSpeed, boolean start) {
9         this.model = model;
10        this.maxSpeed = maxSpeed;
11        this.start = start;
12    }
13
14    public Car(String model, int maxSpeed) { 1개 사용 위치
15        this(model, maxSpeed, start: false);
16    }
17
18    public Car() { 2개 사용 위치
19        this( model: "아반떼", maxSpeed: 200, start: false);
20    }
21
22    void setStart() { 1개 사용 위치
23        start = true;
24    }
25 }
```

클래스

- return 문
 - 메소드의 실행을 강제 종료하고, 호출한 곳으로 돌아 감
 - 메소드 선언에 리턴 타입이 있을 경우 return 문 뒤에 리턴값 지정 필수

```
return [리턴값];
```

클래스

```
Main.java x
1
2
3 public class Main {
4     public static void main(String[] args) {
5         Car myCar1 = new Car();
6
7         myCar1.model = "제네시스";
8         myCar1.maxSpeed = 250;
9         myCar1.start = false;
10
11         Car myCar2 = new Car(model: "소나타", maxSpeed: 220);
12         Car myCar3 = new Car();
13
14         System.out.println("차종: " + myCar1.model);
15         System.out.println("차종: " + myCar2.model);
16         System.out.println("차종: " + myCar3.model);
17
18         int maxSpeed = myCar3.getMaxSpeed();
19         System.out.println("최대 속도: " + maxSpeed);
20
21
22     }
23 }
24
25
26
27
28
29

Car.java x
1
2
3 public class Car { 6개 사용 위치
4     String model; 5개 사용 위치
5     int maxSpeed; 3개 사용 위치
6     boolean start; 3개 사용 위치
7
8     public Car(String model, int maxSpeed, boolean start) {
9         this.model = model;
10        this.maxSpeed = maxSpeed;
11        this.start = start;
12    }
13
14    public Car(String model, int maxSpeed) { 1개 사용 위치
15        this(model, maxSpeed, start: false);
16    }
17
18    public Car() { 2개 사용 위치
19        this(model: "아반떼", maxSpeed: 200, start: false);
20    }
21
22    int getMaxSpeed() { 1개 사용 위치
23        return maxSpeed;
24    }
25
26    void setStart() { 0개의 사용위치
27        start = true;
28    }
29 }
```

클래스

- 가변길이 매개변수

- 메소드가 가변길이 매개변수를 가지고 있다면, 매개변수의 개수와 상관없이 매개값을 줄 수 있음
 - 가변길이 매개변수는 메소드의 매개변수 중 가장 마지막에 위치해야 함

```
int sum(int ... values) {  
}
```

```
int[] values = { 1, 2, 3 };  
int result = sum(values);
```

```
int result = sum(new int[] { 1, 2, 3 });
```


클래스

```
Main.java x
1 public class Main {
2     public static void main(String[] args) {
3         Calculator calculator = new Calculator();
4
5         int result = calculator.sum(...values: 1, 2, 3, 4);
6         System.out.println(result);
7
8         int[] values = {-1, 0, 1, 2};
9         result = calculator.sum(values);
10        System.out.println(result);
11
12        result = calculator.sum(new int[] {1, 2, 3, 4});
13        System.out.println(result);
14    }
15 }
16 }

Calculator.java x
1 public class Calculator { 2개 사용 위치
2     @ int sum(int ... values) { 3개 사용 위치
3         int sum = 0;
4         for (int i=0; i<values.length; i++) {
5             sum += values[i];
6         }
7         return sum;
8     }
9 }
10 }
```

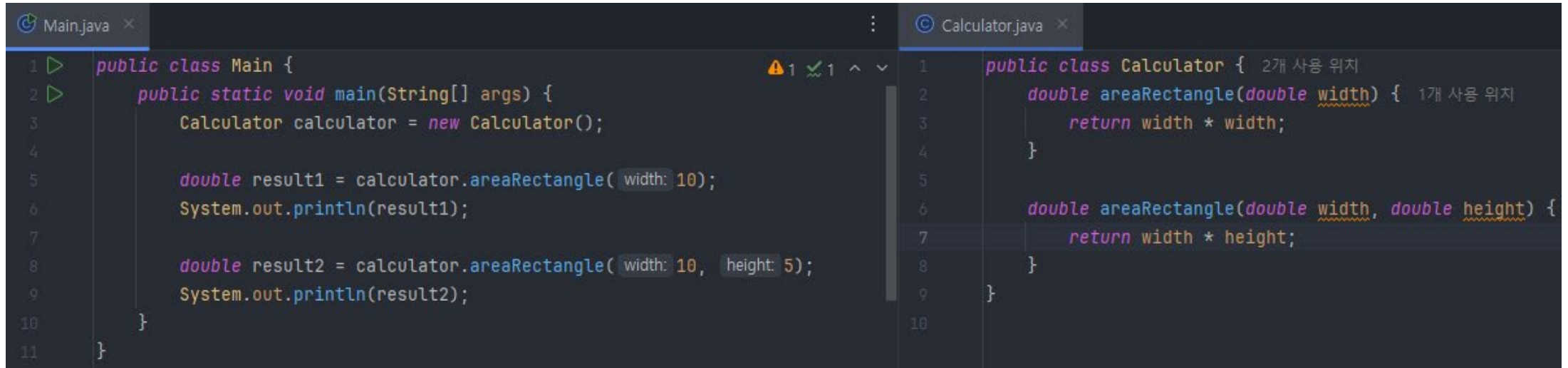
클래스

- 메소드 오버로딩

- 메소드 이름은 같지만, 매개변수의 타입, 개수, 순서가 다른 메소드를 여러 개 선언하는 것

```
class 클래스 {  
    리턴타입 메소드이름 ( 타입 변수, ... ) { ... }  
    ↑      ↑      ↑  
    무관   동일   타입, 개수, 순서가 달라야 함  
    ↓      ↓      ↓  
    리턴타입 메소드이름 ( 타입 변수, ... ) { ... }  
}
```

클래스



```
1 public class Main {
2     public static void main(String[] args) {
3         Calculator calculator = new Calculator();
4
5         double result1 = calculator.areaRectangle(width: 10);
6         System.out.println(result1);
7
8         double result2 = calculator.areaRectangle(width: 10, height: 5);
9         System.out.println(result2);
10    }
11 }
```

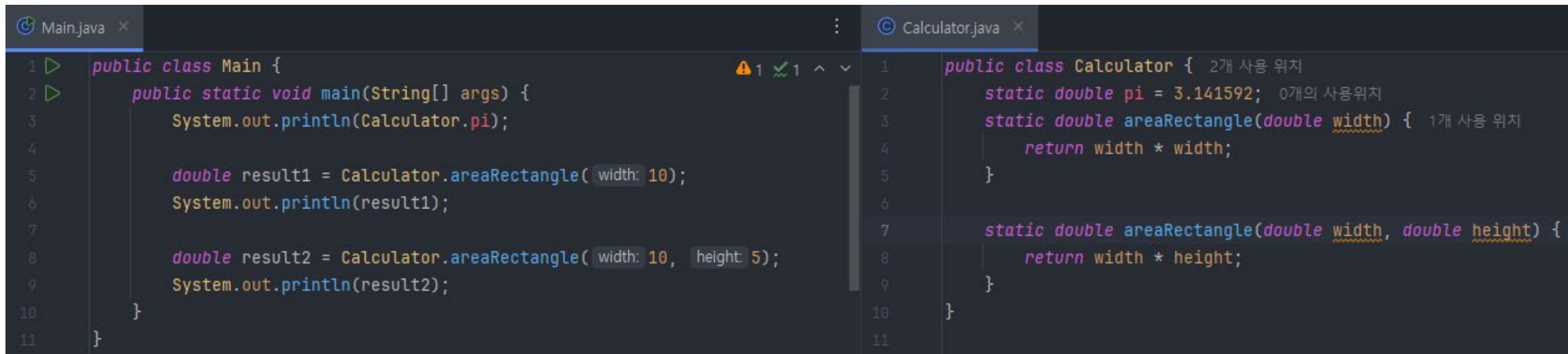
```
1 public class Calculator { 2개 사용 위치
2     double areaRectangle(double width) { 1개 사용 위치
3         return width * width;
4     }
5
6     double areaRectangle(double width, double height) {
7         return width * height;
8     }
9 }
10
```

클래스

- 정적 멤버 선언
 - 정적 멤버: 메소드 영역의 클래스에 고정적으로 위치하는 멤버
 - 정적 멤버는 인스턴스를 생성할 필요 없이 클래스를 통해 바로 사용 가능

```
public class 클래스 {  
    //정적 필드 선언  
    static 타입 필드 [= 초기값];  
  
    //정적 메소드  
    static 리턴타입 메소드( 매개변수, ... ) { ... }  
}
```

클래스



```

Main.java
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(Calculator.pi);
4
5         double result1 = Calculator.areaRectangle( width: 10);
6         System.out.println(result1);
7
8         double result2 = Calculator.areaRectangle( width: 10, height: 5);
9         System.out.println(result2);
10    }
11 }

Calculator.java
1 public class Calculator {
2     static double pi = 3.141592;
3     static double areaRectangle(double width) {
4         return width * width;
5     }
6
7     static double areaRectangle(double width, double height) {
8         return width * height;
9     }
10 }
11
```

클래스

- 정적 블록

- 정적 필드를 선언할 때 복잡한 초기화 작업이 필요하다면 정적 블록 이용
- 정적 블록은 클래스가 메모리로 로딩될 때 자동으로 실행
- 정적 필드는 객체 생성 없이도 사용할 수 있기 때문에 생성자에서 초기화 작업을 하지 않음

```
static {  
    ...  
}
```

클래스

```
Main.java x
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(Calculator.info);
6
7         double result1 = Calculator.areaRectangle( width: 10);
8         System.out.println(result1);
9
10        double result2 = Calculator.areaRectangle( width: 10, height: 5);
11        System.out.println(result2);
12    }
13 }
14
15
16
17

Calculator.java x
1 public class Calculator { 3개 사용 위치
2     static String company = "Samsung"; 1개 사용 위치
3     static String model = "mk1"; 1개 사용 위치
4     static String info; 1개 사용 위치
5     static {
6         info = company + "-" + model;
7     }
8     static double areaRectangle(double width) { 1개 사용 위치
9         return width * width;
10    }
11
12    static double areaRectangle(double width, double height) {
13        return width * height;
14    }
15 }
```

클래스

- 인스턴스 멤버 사용 불가
 - 정적 메소드와 정적 블록은 내부에 인스턴스 필드나 인스턴스 메소드를 사용할 수 없으며 this 도 사용할 수 없음
 - 정적 메소드와 정적 블록에서 인스턴스 멤버를 사용하고 싶다면, 객체를 먼저 생성하고 참조 변수로 접근

```
static void Method3() {  
    //객체 생성  
    ClassName obj = new ClassName();  
    //인스턴스 멤버 사용  
    obj.field1 = 10;  
    obj.method1();  
}
```


클래스

```
Main.java x
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(Calculator.info);
4
5         double result1 = Calculator.areaRectangle(width: 10);
6         System.out.println(result1);
7
8         double result2 = Calculator.areaRectangle(width: 10, height: 5);
9         System.out.println(result2);
10    }
11 }

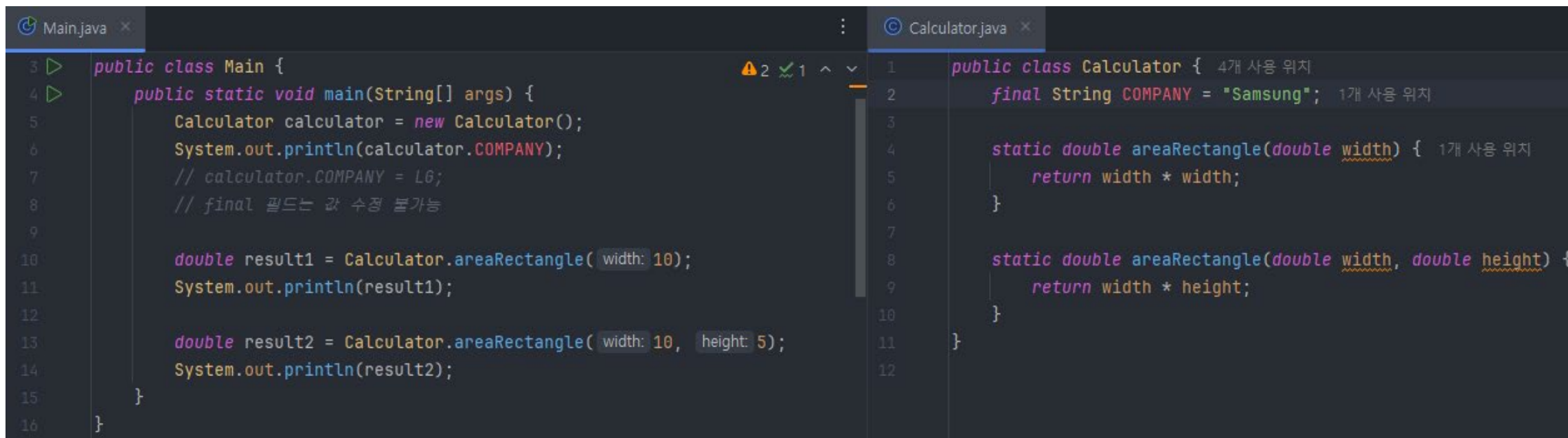
Calculator.java x
1 public class Calculator { 5개 사용 위치
2     String number = "123"; 1개 사용 위치
3     static String company = "Samsung"; 2개 사용 위치
4     static String model = "mk1"; 1개 사용 위치
5     static String info; 2개 사용 위치
6     static {
7         info = company + "-" + model;
8     }
9     static double areaRectangle(double width) { 1개 사용 위치
10        return width * width;
11    }
12
13    static double areaRectangle(double width, double height) { 1개 사용 위치
14        Calculator calculator = new Calculator();
15        System.out.println("모델 번호: " + company + "-" + calculator.number);
16        return width * height;
17    }
18 }
```

클래스

- final 필드 선언
 - final 필드는 초기값이 저장되면 최종적인 값이 되어서 프로그램 실행 도중에 수정할 수 없음
 - final 필드에 초기값을 주려면 필드 선언 시에 초기값을 대입하거나, 생성자에서 초기값을 대입

```
final 타입 필드 [=초기값];
```

클래스



```
3 public class Main {
4     public static void main(String[] args) {
5         Calculator calculator = new Calculator();
6         System.out.println(calculator.COMPANY);
7         // calculator.COMPANY = LG;
8         // final 필드는 값 수정 불가능
9
10        double result1 = Calculator.areaRectangle(width: 10);
11        System.out.println(result1);
12
13        double result2 = Calculator.areaRectangle(width: 10, height: 5);
14        System.out.println(result2);
15    }
16 }

1 public class Calculator { 4개 사용 위치
2     final String COMPANY = "Samsung"; 1개 사용 위치
3
4     static double areaRectangle(double width) { 1개 사용 위치
5         return width * width;
6     }
7
8     static double areaRectangle(double width, double height) {
9         return width * height;
10    }
11 }
12 }
```

클래스

- 상수 선언
 - 상수: 불변의 값을 저장하는 필드
 - 상수는 객체마다 저장할 필요가 없고, 여러 개의 값을 가져도 안 되기 때문에 static 이면서 final

```
static final 타입 상수 [= 초기값];
```

클래스

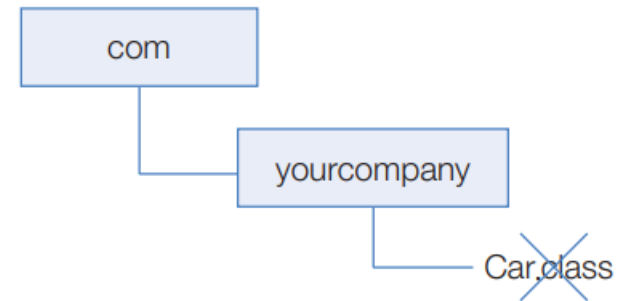
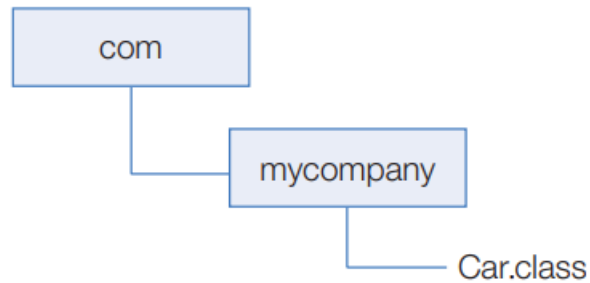
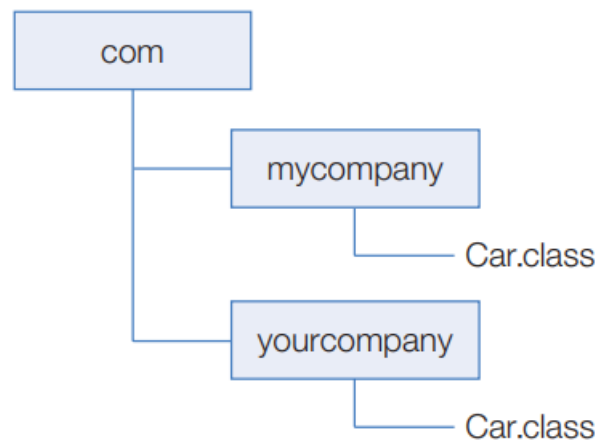
```
Main.java Calculator.java
1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(Calculator.info);
4         System.out.println(Calculator.PI);
5
6         double result1 = Calculator.areaRectangle( width: 10);
7         System.out.println(result1);
8
9         double result2 = Calculator.areaRectangle( width: 10, height: 5);
10        System.out.println(result2);
11    }
12 }
13
14
15
16
17
18
19

1 public class Calculator { 5개 사용 위치
2     static final double PI = 3.14; 0개의 사용위치
3     String number = "123"; 1개 사용 위치
4     static String company = "Samsung"; 2개 사용 위치
5     static String model = "mk1"; 1개 사용 위치
6     static String info; 2개 사용 위치
7     static {
8         info = company + "-" + model;
9     }
10    static double areaRectangle(double width) { 1개 사용 위치
11        return width * width;
12    }
13
14    static double areaRectangle(double width, double height) { 1개 사용 위치
15        Calculator calculator = new Calculator();
16        System.out.println("모델 번호: " + company + "-" + calculator.number);
17        return width * height;
18    }
19 }
```

자바의 패키지

- 패키지

- 클래스의 일부분이며, 클래스를 식별하는 용도
- 상위 패키지와 하위 패키지를 도트 (.) 로 구분
- 패키지에 속한 바이트코드 파일 (.class)은 따로 떼어내어 다른 디렉토리로 이동할 수 없음



자바의 패키지

- 패키지 선언

- 패키지 선언은 `package` 키워드와 함께 패키지 이름을 기술한 것
- 항상 소스 파일 최상단에 위치

```
package 상위패키지.하위패키지;
```

```
public class 클래스명 { ... }
```

- 패키지 이름은 모두 소문자로 작성, 패키지 이름이 서로 중복되지 않도록 회사 도메인 이름의 역순으로 작성, 마지막에는 프로젝트 이름

```
com.samsung.projectname
```

```
com.lg.projectname
```

```
org.apache.projectname
```

자바의 패키지

- import 문
 - 다른 패키지에 있는 클래스를 사용하려면 import 문을 이용해서 어떤 패키지의 클래스를 사용하는지 명시
 - import 문은 패키지 선언과 클래스 선언 사이에 작성

```
package com.mycompany;
```

Car 클래스의 패키지

```
import com.hankook.Tire;
```

Tire 클래스의 전체 이름

```
public class Car {  
    //필드 선언 시 com.hankook.Tire 클래스를 사용  
    Tire tire = new Tire();  
}
```


자바의 패키지

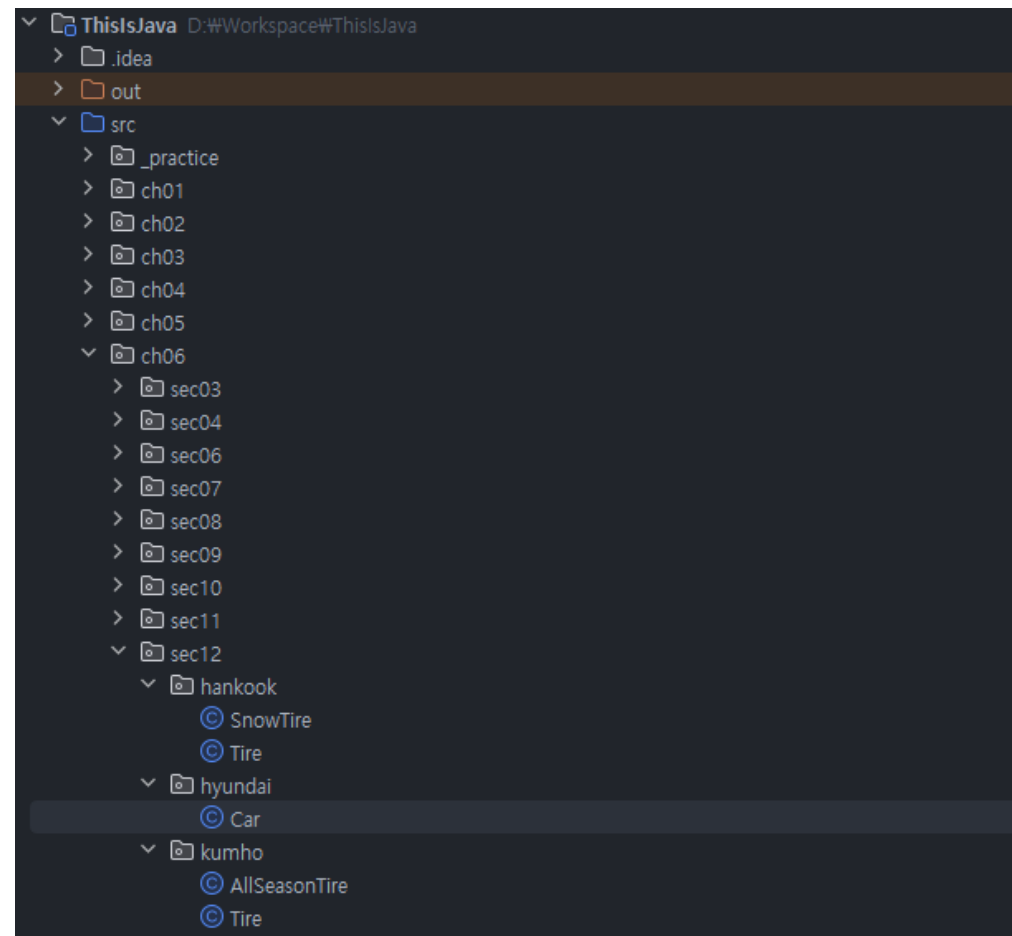
- 패키지

- src 패키지를 시작으로 경로 읽기

```
package ch06.sec12.hyundai;

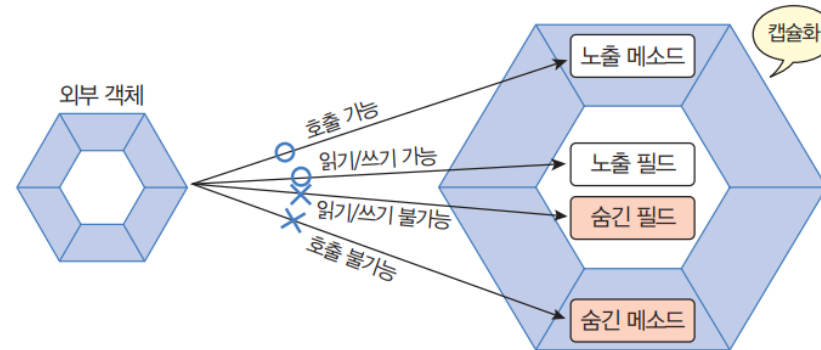
//import 문으로 다른 패키지 클래스 사용을 명시
import ...

public class Car {
    //부품 필드 선언
    ch06.sec12.hankook.Tire tire1 = new ch06.sec12.hankook.Tire(); 0개의 사용위치
    ch06.sec12.kumho.Tire tire2 = new ch06.sec12.kumho.Tire(); 0개의 사용위치
    SnowTire tire3 = new SnowTire(); 0개의 사용위치
    AllSeasonTire tire4 = new AllSeasonTire(); 0개의 사용위치
}
```

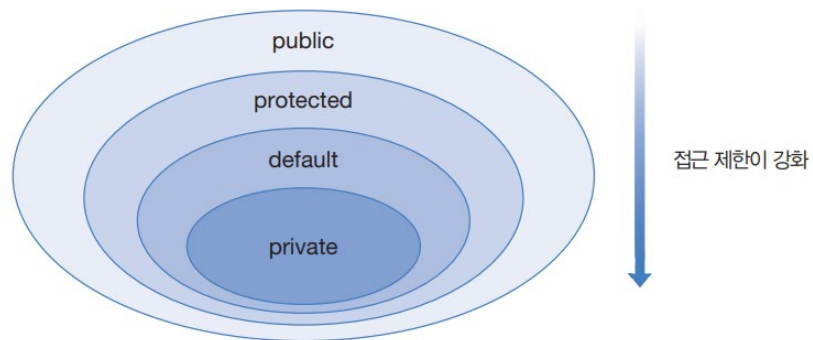


자바의 패키지

- 접근 제한자
 - 중요한 필드와 메소드가 외부로 노출되지 않도록 함



- 접근 제한자: public, protected, private

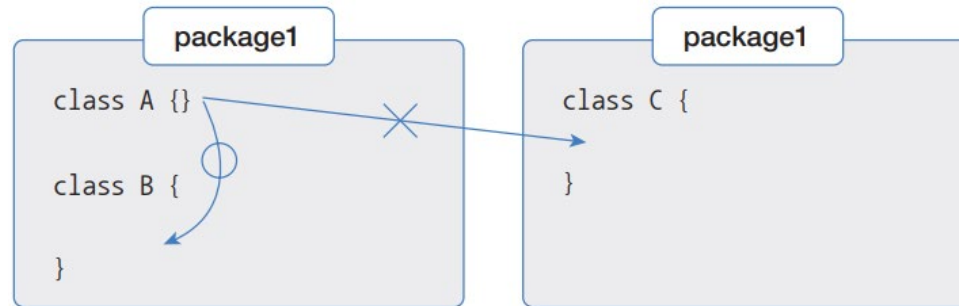


접근 제한자	제한 대상	제한 범위
public	클래스, 필드, 생성자, 메소드	없음
protected	필드, 생성자, 메소드	같은 패키지이거나, 자식 객체만 사용 가능 (7장 상속에서 자세히 설명)
(default)	클래스, 필드, 생성자, 메소드	같은 패키지
private	필드, 생성자, 메소드	객체 내부

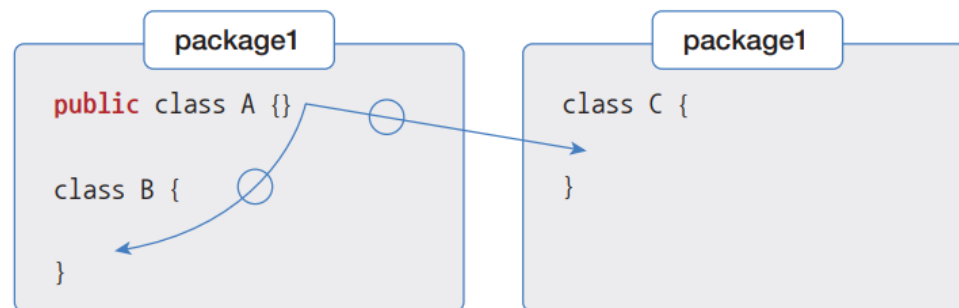
자바의 패키지

- 클래스의 접근 제한

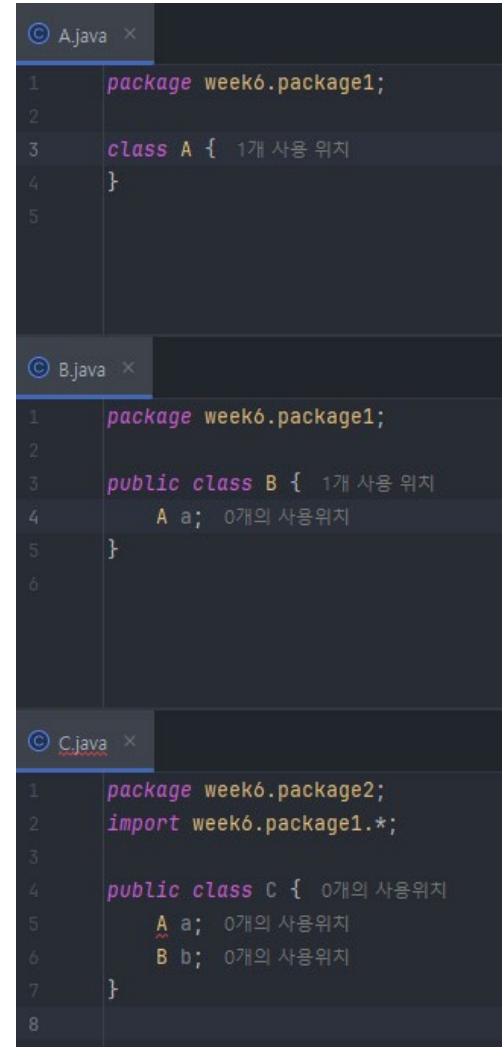
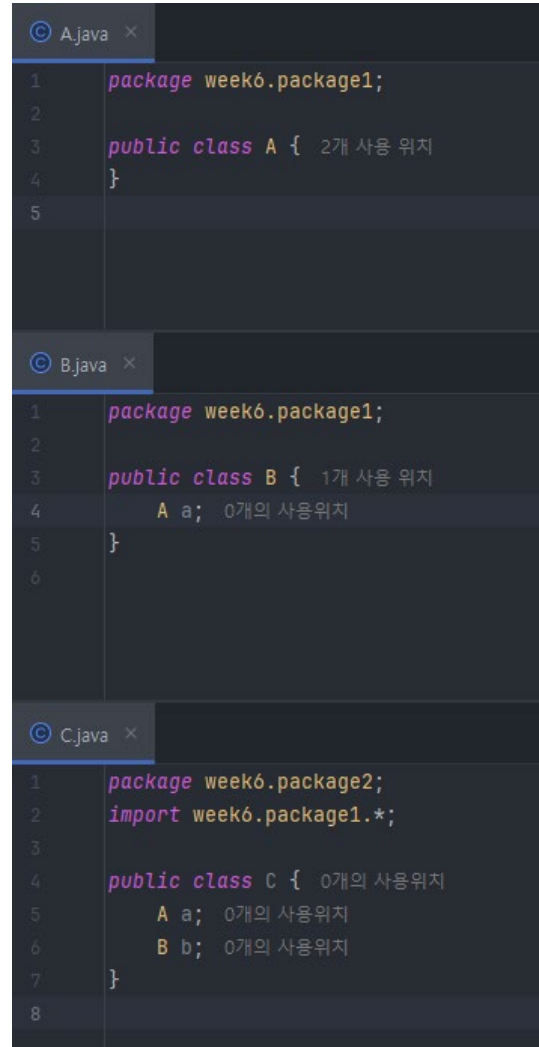
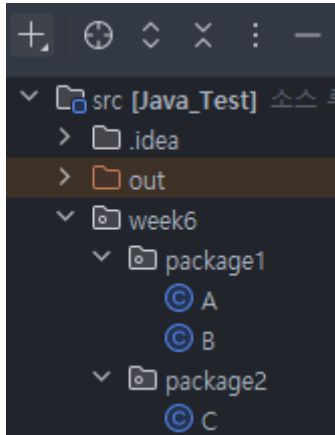
- public 접근 제한자를 생략하면, 다른 패키지에서 사용할 수 없음



- public 접근 제한자를 붙이면, 다른 패키지에서도 사용할 수 있음



자바의 패키지



자바의 패키지

- 생성자의 접근 제한

- 생성자는 public, default, private 접근 제한을 가질 수 있음

```
public class ClassName {  
    //생성자 선언  
    [ public | private ] ClassName(...) { ... }  
}
```

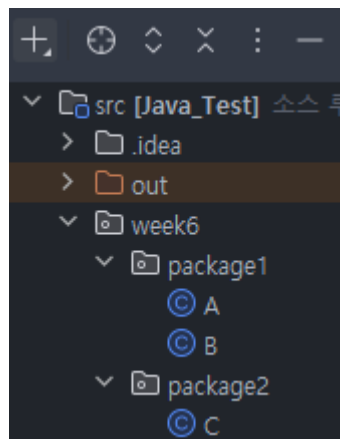
접근 제한자	생성자	설명
public	클래스(...)	모든 패키지에서 생성자를 호출할 수 있다. = 모든 패키지에서 객체를 생성할 수 있다.
	클래스(...)	같은 패키지에서만 생성자를 호출할 수 있다. = 같은 패키지에서만 객체를 생성할 수 있다.
private	클래스(...)	클래스 내부에서만 생성자를 호출할 수 있다. = 클래스 내부에서만 객체를 생성할 수 있다.

자바의 패키지

```
A.java
1 package week6.package1;
2
3 public class A {
4     A a1 = new A(b: true);
5     A a2 = new A(i: 1);
6     A a3 = new A(s: "string");
7
8     public A(boolean b) {}
9     A(int i) {}
10    private A(String s) {}
11 }

B.java
1 package week6.package1;
2
3 public class B {
4     A a1 = new A(b: false);
5     A a2 = new A(i: 3);
6     A a3 = new A("STRING");
7 }

C.java
1 package week6.package2;
2 import week6.package1.*;
3
4 public class C {
5     A a1 = new A(b: true);
6     A a2 = new A(b: 3);
7     A a3 = new A(b: "문자열");
8 }
9
```



접근 제한자	제한 대상	제한 범위
public	클래스, 필드, 생성자, 메소드	없음
protected	필드, 생성자, 메소드	같은 패키지이거나, 자식 객체만 사용 가능 (7장 상속에서 자세히 설명)
(default)	클래스, 필드, 생성자, 메소드	같은 패키지
private	필드, 생성자, 메소드	객체 내부

자바의 패키지

- 필드와 메소드의 접근 제한
 - 필드와 메소드는 public, default, private 접근 제한을 가질 수 있음

```
//필드 선언  
[ public | private ] 타입 필드;  
  
//메소드 선언  
[ public | private ] 리턴타입 메소드(...) { ... }
```

접근 제한자	생성자	설명
public	필드 메소드(...)	모든 패키지에서 필드를 읽고 변경할 수 있다. 모든 패키지에서 메소드를 호출할 수 있다.
	필드 메소드(...)	같은 패키지에서만 필드를 읽고 변경할 수 있다. 같은 패키지에서만 메소드를 호출할 수 있다.
private	필드 메소드(...)	클래스 내부에서만 필드를 읽고 변경할 수 있다. 클래스 내부에서만 메소드를 호출할 수 있다.

자바의 패키지

```
A.java
1 package week6.package1;
2
3 public class A {
4     public int field1;
5     int field2;
6     private int field3;
7
8     public void method1() {}
9     void method2() {}
10    private void method3() {}
11
12    public A() {
13        field1 = 1;
14        field2 = 2;
15        field3 = 3;
16        method1();
17        method2();
18        method3();
19    }
20 }
21

B.java
1 package week6.package1;
2
3 public class B {
4     public B() {
5         A a = new A();
6         a.field1 = 1;
7         a.field2 = 2;
8         a.field3 = 3;
9
10        a.method1();
11        a.method2();
12        a.method3();
13    }
14 }
15

C.java
1 package week6.package2;
2 import week6.package1.*;
3
4 public class C {
5     public C() {
6         A a = new A();
7         a.field1 = 1;
8         a.field2 = 2;
9         a.field3 = 3;
10
11        a.method1();
12        a.method2();
13        a.method3();
14    }
15 }
16
17
```


자바의 패키지

- Getter 와 Setter

- Getter: 외부에서 직접적인 필드 접근을 막고, 메소드를 통해 필드에 접근
- Setter: 데이터를 검증해서 유효한 값만 필드에 저장하는 메소드

```
private 타입 fieldName;
```

필드 접근 제한자: private

```
//Getter
```

```
public 타입 getFieldName() {  
    return fieldName;  
}
```

접근 제한자: public

리턴 타입: 필드타입

메소드 이름: get + 필드이름(첫 글자 대문자)

리턴값: 필드값

```
//Setter
```

```
public void setFieldName(타입 fieldName) {  
    this.fieldName = fieldName;  
}
```

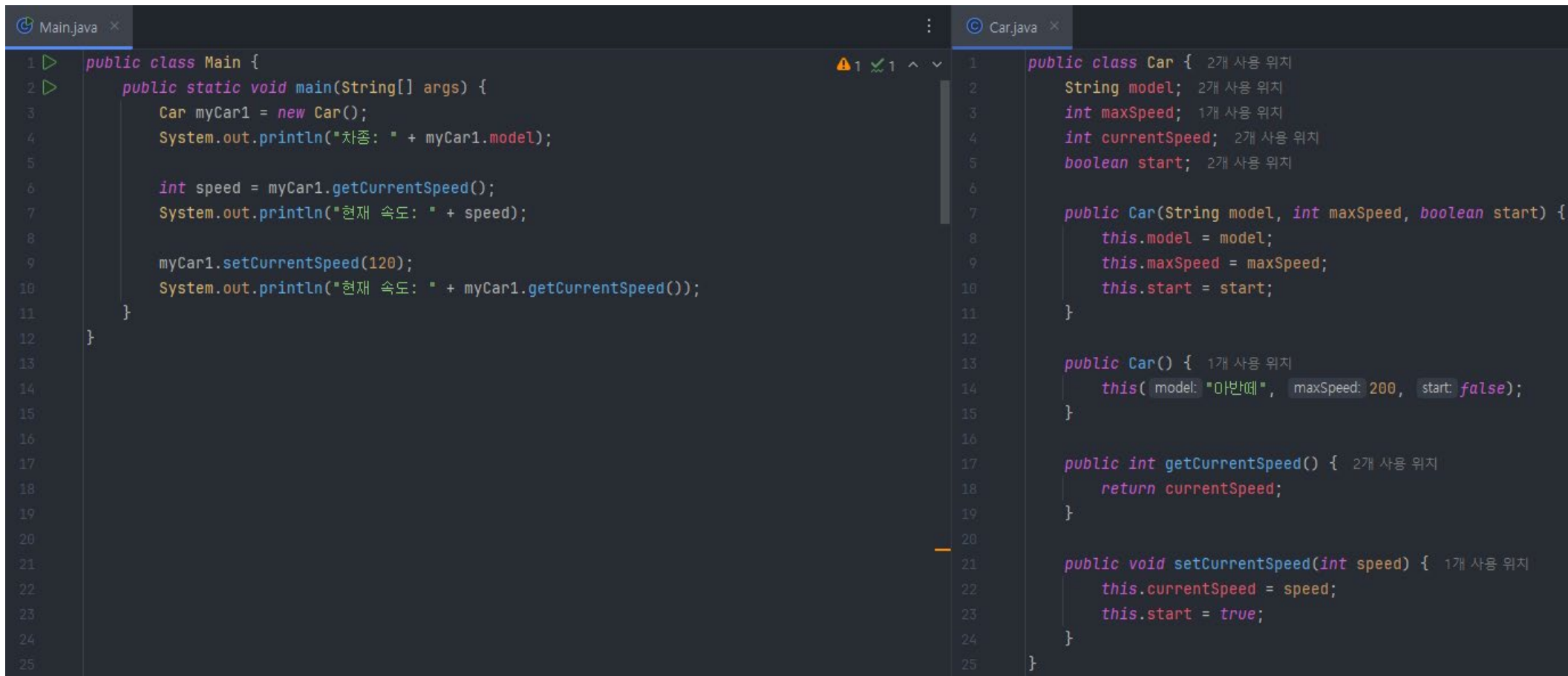
접근 제한자: public

리턴 타입: void

메소드 이름: set + 필드이름(첫 글자 대문자)

매개변수 타입: 필드타입

자바의 패키지



```
1 public class Main {
2     public static void main(String[] args) {
3         Car myCar1 = new Car();
4         System.out.println("차종: " + myCar1.model);
5
6         int speed = myCar1.getCurrentSpeed();
7         System.out.println("현재 속도: " + speed);
8
9         myCar1.setCurrentSpeed(120);
10        System.out.println("현재 속도: " + myCar1.getCurrentSpeed());
11    }
12 }
13
14
15
16
17
18
19
20
21
22
23
24
25
```

```
1 public class Car { 2개 사용 위치
2     String model; 2개 사용 위치
3     int maxSpeed; 1개 사용 위치
4     int currentSpeed; 2개 사용 위치
5     boolean start; 2개 사용 위치
6
7     public Car(String model, int maxSpeed, boolean start) {
8         this.model = model;
9         this.maxSpeed = maxSpeed;
10        this.start = start;
11    }
12
13    public Car() { 1개 사용 위치
14        this(model: "아반떼", maxSpeed: 200, start: false);
15    }
16
17    public int getCurrentSpeed() { 2개 사용 위치
18        return currentSpeed;
19    }
20
21    public void setCurrentSpeed(int speed) { 1개 사용 위치
22        this.currentSpeed = speed;
23        this.start = true;
24    }
25 }
```

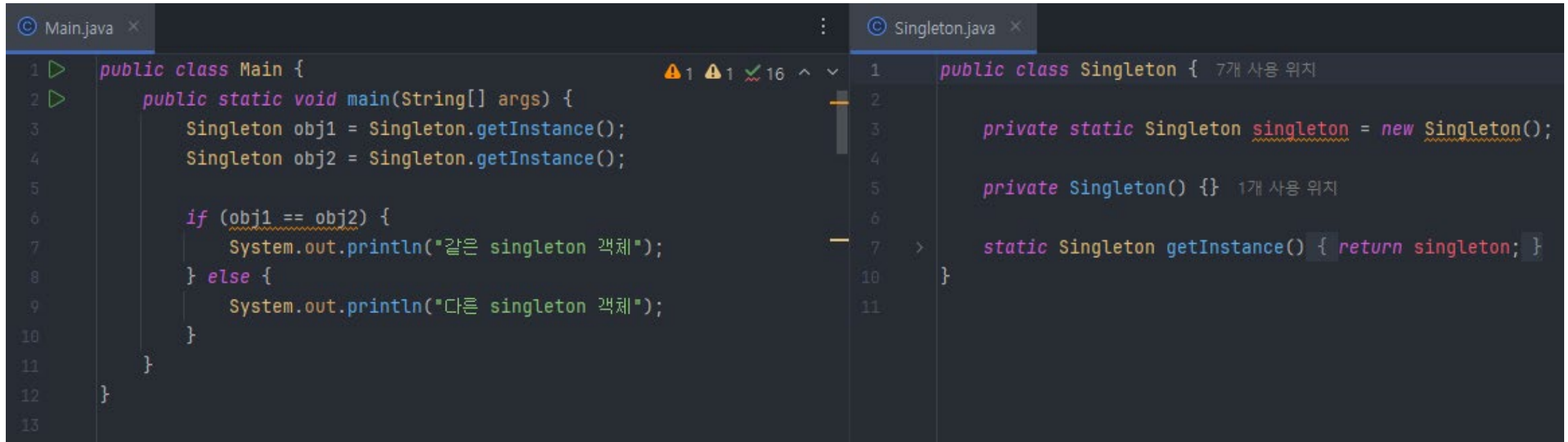
자바의 패키지

- 싱글톤 패턴

- 단 한 개의 객체만 생성해서 사용하고 싶은 경우, 생성자를 private 접근 제한해서 외부에서 new 연산자로 생성자를 호출할 수 없도록 막음
- 대신, 싱글톤 패턴이 제공하는 정적 메소드를 통해 간접적으로 객체 접근

```
public class 클래스 {  
    //private 접근 권한을 갖는 정적 필드 선언과 초기화  
    private static 클래스 singleton = new 클래스(); •----- ①  
  
    //private 접근 권한을 갖는 생성자 선언  
    private 클래스() {}  
  
    //public 접근 권한을 갖는 정적 메소드 선언  
    public static 클래스 getInstance() { •----- ②  
        return singleton;  
    }  
}
```

자바의 패키지



```
1 public class Main {
2     public static void main(String[] args) {
3         Singleton obj1 = Singleton.getInstance();
4         Singleton obj2 = Singleton.getInstance();
5
6         if (obj1 == obj2) {
7             System.out.println("같은 singleton 객체");
8         } else {
9             System.out.println("다른 singleton 객체");
10        }
11    }
12 }
13
```

```
1 public class Singleton { 7개 사용 위치
2
3     private static Singleton singleton = new Singleton();
4
5     private Singleton() {} 1개 사용 위치
6
7     static Singleton getInstance() { return singleton; }
10 }
11
```

